PEA：保护工程助手

完整架构设计与实施规格（v0.2.2）

文档版本： 0.2.2

研究日期： 2026-10-13

状态： 可作为 MVP 实施基线；保护后端仍需逐个交付

目标读者： 开源维护者、软件保护工程师、MCP/Agent 开发者、构建与发布工程师

0. 执行摘要

PEA（Protective Engineering Assistant）是面向自有或获授权软件项目的保护工程编排系统。它把 AI Agent 的需求理解与研究能力、可审核的项目目录、确定性的策略门控、受控构建和自动验证连接起来。PEA 不把“混淆”当作万能答案，也不承诺任意程序无法被逆向；它要回答的是：目标资产实际暴露在哪里、现成方案哪一个适配、采用后能验证什么、还有哪些风险没有消除。

PEA 的首要研发顺序是：

1. 建立证据驱动的保护器目录：从官方仓库、官方文档、可信软件包元数据检索候选，并核对来源、版本、许可证、平台、维护状态和测试证据。
2. 建立分析能力—暴露面—防护方案映射器：参考 REA 等分析框架公开描述的字符串/符号、调用关系、反编译、应用资源图、运行时观察和构建比较能力，检查自有发布物中的对应暴露面。
3. 生成可审阅计划：提供推荐方案、替代方案、排除原因、计划改动、命令摘要、权限、副作用、验证门槛和回滚方式。
4. 经明确批准后受控执行：仅执行经过登记的适配器，以结构化参数、固定版本、最小权限和隔离工作区执行。
5. 验证功能与保护效果：用回归/差分测试、兼容性、性能、产物泄露检查和受控的可读性评估验证结果。
6. 仅在现成工具不适用时研究自研方案：先评估把敏感逻辑移出不可信客户端；对于必须离线运行的受限纯函数，才进入自研 IR/VM 概念验证。

用户提供的 lang-vmp-guide 是 PEA 的重要方法论参考，特别适合用来构建“解释语言 VMP 研究技能”。它不应被不加辨别地视为已验证的通用实现，也不应让其中依赖 Bash、Linux 内核或特定编译器的结论直接推广到其他平台。PEA 会保留其中可迁移的验证方法，并将未经跨平台实测的强度声明转化为待验证假设。

1. 产品定位、目标与非目标

1.1 目标

- 为用户自有或获明确授权的项目选择、集成并验证保护方案。
- 支持只读研究、离线计划、经审批执行和保护效果审计四种模式。
- 将 CLI 与 MCP 作为统一服务层的两个接口；LLM 负责解释和提出方案，确定性策略引擎负责权限、硬约束和发布门槛。
- 目录记录可追溯证据，而不只是项目名称：URL、版本/tag/commit、许可证证据、维护信号、目标平台、支持能力、已验证场景、最后检查时间和已知局限。
- 对发布工件做“发现—修复—复验”闭环，防止构建成功被误判为安全成功。
- 通过版本化的 Agent Skill 指导检索、VMP 可行性评估、集成和验证流程。

1.2 非目标

- 不保证代码绝对不可逆向、密钥在不可信客户端绝对不可提取或反调试能够抵挡拥有完全设备控制权的对手。

- 不破解第三方许可证、不盗用密钥、不修补第三方产品授权逻辑、不规避终端安全软件、不提供隐蔽注入/持久化/恶意载荷隐藏能力。
- MVP 不自动把任意源码或任意二进制转成 VM 字节码，也不将逆向工具导出的伪代码直接作为自研保护器输入。
- 不因为“没有现成项目”就自动执行未知仓库、下载并运行不可信脚本，或在来源不明的代码上进行自我复制式学习。
- 不用反分析手段替代真实的秘密管理、服务器端授权、签名更新或安全设计。

1.3 成功标准

MVP 的成功不是“生成了多少保护代码”，而是：

- 能可靠地找到适配候选并解释为什么匹配或为什么排除；
- 未经明确批准不会发生下载、执行、联网、部署或源文件修改；
- 每次运行能够还原输入版本、工具版本、配置、产物摘要、测试结果和决策证据；
- 保护后核心功能测试、兼容性和性能指标有明确结果；
- “未知”被明确标记为未知，不被视作通过；
- 无合适方案时可以安全地得出“暂不支持”，而不是强行输出未经验证的后端。

2. 核心对象与术语

2.1 资产（Asset）

需要降低暴露风险的具体对象，包括敏感算法、业务规则、授权决策、客户端资源、脚本、协议处理代码、密钥材料、构建与发布链路。资产必须由用户确认；AI 可以建议候选，但不能仅凭函数名自动决定“该函数必须虚拟化”。

2.2 暴露面（Exposure Surface）

软件交付或执行期间可能被观察、提取、修改或重建的界面。例如符号和调试数据、静态字符串、调用与控制流、源映射、应用包资源、IPC/插件接口、网络响应、进程参数、运行期数据、配置与更新机制。

2.3 防护目标（Protection Objective）

应写成可测试的结果，而非抽象形容词。例如：发布产物不包含长期服务端密钥；生产 APK 不包含未获准的调试映射文件；授权决定必须在服务器端执行；保护后的函数对已知测试集保持语义一致；受控篡改会导致签名/完整性验证失败。

2.4 逻辑抽取与函数级虚拟化

“逻辑抽取”在 PEA 中优先指把秘密或高价值决策移到用户控制的服务端、受信任进程或硬件安全边界，而不是从未知第三方二进制提取逻辑。

“函数级虚拟化”是后期研究能力：把用户拥有、可测试、满足受限类型/副作用约束的函数转换为受限 IR，由独立且经过测试的解释器执行。它提高静态阅读成本，但不能消除运行时输入、输出、内存和调用行为暴露。

3. 设计原则

1. 先复用，后自研。 必须先提供检索记录和候选排除理由，再提出自研后端。
2. 按风险路径防护，而非堆叠术语。 每个控制都必须说明缓解哪种风险、剩余什么风险、怎么验证。
3. 证据与推断分离。 静态语法或配置存在不等于运行时必然执行；没有观测不得写成已验证。
4. 平台差异显式化。 REA 的某个 provider 在某系统可用，不代表同一操作在另一个系统、工具版本或授权模式可用。
5. LLM 不是安全边界。 模型提出建议，代码决定规则；模型不能批准自己提出的执行计划。
6. 最小权限与可撤销。 默认只读；所有有副作用的操作均需结构化参数、审批和回滚路径。
7. 先测再承诺。 不能用构建成功替代语义正确性，也不能用“反编译器读不出来”替代安全证明。
8. 许可证和供应链属于硬门槛。 未知许可、来源不清或不兼容的依赖不可因为评分高而进入生产执行。
9. 输出可复现。 记录工具版本、参数、输入/输出摘要、随机种子（若有）、测试和限制。

10. 透明失败。 失败要提供原因和可恢复步骤；不得悄悄切换 provider、降级测试或把 unknown 转成 pass。

4. 总体架构

```
用户 / IDE Agent / CI
        | 自然语言目标或结构化请求
        ▼
CLI + MCP API（统一 service layer）
        |
        ├── Intake & Authorization: 授权、路径范围、执行模式
        ├── Project Profiler: 语言、构建链、测试与工件清单
        ├── Exposure Mapper: 分析能力到发布暴露面的映射
        ├── Research Agent: 搜索官方仓库、文档、包元数据、论文
        ├── Catalog Resolver: 候选收录、许可/维护/版本/平台硬约束
        ├── Strategy Engine: 硬约束过滤 + 多目标评分
        ├── Approval Gate: 冻结计划摘要、权限、副作用和文件清单
        ├── Adapter Runner: 隔离工作区、命令白名单、资源/网络限制
        ├── Verification Engine: 等价性、兼容性、性能、泄露与签名检查
        └── Evidence Ledger: 证据、工件摘要、测试、失败原因和回滚
                  |
                  ▼
 SQLite Catalog / Local Artifact Store / Audit Log / Reports
```

4.1 技术栈建议

层	MVP 推荐	设计约束
语言	Python 3.12+	标准库原型可离线运行；生产依赖锁版本
接口	CLI + MCP Server	两者调用相同 service layer，不复制业务规则
Schema	JSON Schema 2020-12	输入输出强校验；未知字段和限制值策略显式定义
目录	SQLite + JSON 导入导出	离线可用；在线结果先入待审核队列
执行器	argv 数组 + 专用 adapter	禁止模型生成 shell 字符串直接执行
隔离	临时工作区、低权限用户、容器/沙箱	默认断网、资源限制、清理子进程树
验证	unittest/pytest + fixture corpus	真实 provider 测试与 mock 测试分层标注
报告	Markdown + JSON + SBOM/许可清单	可追溯、去敏、明确未知与覆盖范围

5. 用户模式与完整流程

5.1 四种运行模式

- Research-only（只读研究）：搜索项目和文档，不能执行候选项目、修改文件或联网以外传用户代码。
- Plan-only（仅生成计划）：读取最少必要的项目元数据，输出计划、变更预览和测试门槛，不执行保护器。
- Approved-apply（经审批执行）：只执行绑定到指定计划摘要、输入工件摘要、工具版本和工作区的短时一次性批准。
- Audit-only（保护审计）：检查用户指定的发布工件，优先不执行目标程序；需要动态测试时必须另外审批和隔离。

5.2 标准工作流

1. 确认所有权/书面授权与目标范围。
2. 读取项目清单、构建配置和测试入口，不默认执行项目脚本。
3. 识别目标语言、运行时、平台、发布物和可测试范围。
4. 通过 Exposure Mapper 生成暴露面盘点，给出证据位置和置信度。

5. 生成威胁模型：资产价值、攻击者权限、联网/离线要求、性能/体积预算、兼容边界。
6. 先查询本地目录；在用户允许的网络策略下检索官方源。记录查询时间和候选原始来源。
7. 执行硬约束过滤，明确列出不适用和排除原因，再对余下候选评分。
8. 生成不超过三个可比较方案：低风险、平衡、高保护成本，并包括不做自动保护的选项。
9. 对候选执行能力/许可/来源审查；代码下载与脚本运行是分开的审批行为。
10. 冻结计划，包括文件范围、命令数组、输出路径、网络需求、资源上限、工具版本、验证与回滚。
11. 用户批准后在临时工作区执行；原始工作树保持不变。
12. 运行语义/回归、兼容性、性能、产物泄露与分发检查。
13. 输出报告；仅在所有强制门槛通过且用户确认后，生成可发布候选。

5.3 无现成方案的分支

```
当前目录是否存在满足硬约束的成熟方案?
  ├── 有：锁定版本 → 在 fixture / 样例项目验证 → 再对用户项目做 dry-run
  └── 无：
      ├── 敏感内容能否移出不可信客户端?
      │     ├── 能：生成服务端/可信边界设计和 API 契约
      │     └── 不能：
      │           ├── 是否为用户拥有、可测试、纯函数或受限脚本范围?
      │           │     ├── 否：停止自动变换，提供人工工程方案
      │           │     └── 是：创建独立 PoC，定义 IR/解释器范围与拒绝规则
      │           └── 差分、异常、模糊、资源和性能门槛全通过才评估发布
      └── 证据不足时输出 blocked/unknown，而不是自动下载/执行未知工具
```

6. REA 式分析能力映射器

PEA 把 REA 看作一个公开可观察的分析能力参考，而不是需要“封杀”的特定工具。能力目录必须在运行时从工具/provider catalog 与当前版本信息中确认，不可把博客、旧 README 或旧工具清单当作永久事实。

分析面	典型观察能力	PEA 检查与保护方向	验证证据
静态工件识别	文件/包类型、节区、元数据、哈希、资源清单	确认发布清单、清除无用开发文件、校验资源签名和摘要	工件 manifest、重现构建 diff
字符串、符号、函数目录	字符串搜索、名称/符号列表、过程枚举	搜索硬编码密钥、内部 URL、调试路径；裁剪不必要符号	发布物扫描报告与人工复核
调用/引用关系	xrefs、callers/callees、CFG、调用图	检查关键职责是否暴露过多；先评估架构迁移，再选择性混淆/VMP	保护范围清单、保护前后静态观察差异
伪代码与汇编	函数反编译、批量反编译、汇编查看	对少量高价值、可独立且可测试函数评估成熟保护器	工具/版本/样本固定的可复现分析报告
JS/Electron 应用图	imports、bundle 模块、source map、preload、contextBridge、IPC	控制 source map 发布；最小化 preload/IPC；对 sender/origin/schema 做服务端或主进程校验	工件扫描、IPC 契约测试、Electron 集成测试
浏览器/网络观察	页面行为、接口响应、请求元数据	不在客户端保留长效秘密；服务器端授权、最小响应、短期令牌和速率限制	API 契约测试、访问控制负向测试
进程/运行时观察	进程生命周期、输出、文件/网络活动（依 provider 而定）	避免秘密进入 argv、stdout/stderr、日志、core dump；限制服务进程权限	沙箱运行报告、日志脱敏测试
.NET / Android / 托管代码	元数据、程序集/DEX、资源、映射文件、反射行为	评估成熟混淆与裁剪；保管 mapping 文件；测试反射/序列化/JNI	平台集成测试、mapping 访问策略

分析面	典型观察能力	PEA 检查与保护方向	验证证据
比较与证据	snapshots、artifact diff、function compare、Evidence	建立发布基线，确认保护前后语义及产物差异	Evidence ID、工件摘要、测试结果和 coverage

6.1 保护策略的判定规则

- 密钥或高价值决策：首先检验是否可移到服务器端/可信边界；混淆不是秘密存储。
- 只减少随手阅读：平台原生裁剪、符号清理、适度混淆往往更经济。
- 必须离线的脚本/解释语言：将 lang-vmp-guide 用作研究技能，先识别语法/词法/解释器边界和测试可行性；不默认需要 VMP。
- 必须离线的原生高价值函数：先评估成熟 VMP/混淆产品和 ABI、异常、浮点、内联限制；只对经核验的函数范围尝试。
- 防篡改：优先签名、可信更新、服务器端校验与可撤销许可；本地完整性检测是附加信号，不可单独作为信任根。
- 无法明确验证：报告 unknown，不得把“没有发现”描述为“证明安全”。

7. Research Agent 与目录设计

7.1 信任层级

1. 一级源：项目官方仓库、官方文档、官方发行包、正式许可证文本、平台供应商文档。
2. 二级源：可信包管理元数据、维护者发布说明、可复核的安全公告和测试结果。
3. 发现源：搜索引擎、论坛、博客、问答、视频、社区帖子。仅用于发现，不能单独证明许可、维护状态或安全属性。

每个重要字段需有独立的 evidence 记录：URL、取得时间、内容摘要/commit、支持的结论、证据等级、需要人工复核的点。搜索摘要不得直接覆盖官方 LICENSE 文件的结论。

7.2 候选项目检索步骤

1. 将请求拆成目标、语言/版本、目标平台、保护属性、离线约束、构建集成方式、性能预算和许可要求。
2. 生成搜索词，但不执行候选仓库脚本。
3. 获取官方仓库/发行说明/许可证/安装文档，优先使用 release/tag 或 commit SHA。
4. 核验近期维护信号、支持的输入/输出、系统要求、依赖、数据外传行为和已知限制。
5. 对归档、未知许可证、无签名来源、不可复现构建、平台不匹配候选执行硬排除或进入研究队列。
6. 在隔离环境使用自有 fixture 验证其最小能力；记录失败样例，而非只保留成功结果。
7. 只有经过批准才把候选从 discovered 提升为 reviewed、fixture_verified 或 production_eligible。

7.3 项目目录字段

每个条目至少包含：id、名称、类别、官方 URL、文档 URL、仓库地址、release/tag/commit、下载来源、SHA-256 或签名证据、SPDX 许可证及许可证状态、维护状态、目标语言/平台、能力清单、支持/不支持矩阵、所需权限、副作用、网络需求、数据处理、依赖摘要、验证工件、最后核验时间、已知局限、评分细节、人工审批人。

状态建议：

- discovered：只发现，还未核验；不可执行。
- reviewed：来源、许可和适用范围已审阅；不能代表运行成功。
- fixture_verified：通过 PEA 自有 fixture 的特定验证。
- project_verified：通过目标用户项目的已批准验证。
- production_eligible：许可、供应链、平台、测试和运维门槛全部通过。
- blocked / archived / unknown：不可自动推荐为生产执行项。

8. 评分与策略选择

先执行硬约束过滤，再评分。硬约束包括授权、许可证、目标平台、工具来源完整性、执行权限、性能红线和不可接受的副作用。任何一项失败不得由综合分补偿。

建议评分（通过硬门槛后，0–5 分）：

维度	初始权重	依据
功能与目标匹配	25%	所需保护属性是否有对应能力及真实测试
平台/运行时适配	20%	精确平台、架构、运行时版本覆盖
正确性证据	15%	官方测试、PEA fixture、项目集成测试分开记分
许可证与供应链可信度	15%	license 文件、发行物来源、签名/摘要、依赖可审计性
维护和升级情况	10%	release 活动、已知问题、更新策略
性能/体积预算	10%	实测数据与项目预算
可运维性	5%	回滚、诊断、故障定位和工具输出

评分只用于同类可行候选之间排序。候选排序需要展示每个维度和证据缺口，不能只显示一个总分。

9. 组件与 MCP 工具合同

9.1 建议核心工具

工具	副作用	主要合同
pea.assess_project	只读文件访问	在授权根目录内盘点构建元数据、测试和候选资产，不执行项目脚本
pea.map_exposure	只读扫描	输入目标平台与工件 manifest，输出暴露面、位置、证据与不确定性
pea.search_catalog	可选网络读取	查询本地目录或经批准的官方源；线上候选进入待审队列
pea.inspect_candidate	可选网络读取/工件读取	读取许可证、版本、支持矩阵、依赖、安装动作和风险，不执行安装脚本
pea.plan_protection	无	生成候选比较、执行前检查、文件清单、命令摘要、测试门槛与回滚方案
pea.review_plan	写审批状态	由用户明确确认并生成绑定 plan digest 的一次性短期批准
pea.apply_plan	写临时工作区/启动进程	执行固定 adapter 的结构化参数；不接受任意 shell 文字
pea.verify_output	可启动测试进程	运行用户已审阅的测试剖面，返回 pass/fail/unknown 与证据
pea.compare_results	只读	比较原始/保护版行为、性能和暴露面差异
pea.export_report	写输出文件	导出去敏报告、清单和回滚信息，尊重覆盖策略
pea.rollback	删除 PEA 临时数据/撤销自有补丁	只撤销本次创建的资源，不运行不受控的 reset

每个工具都必须带 side_effects、所需权限、目标范围、超时、取消语义、输出大小上限和可预测的错误码。MCP 标注应从同一份 effect/capability 定义生成，避免文档与真实工具不一致。

9.2 批准令牌

批准必须绑定：plan_id、plan_digest、项目根目录的规范路径、输入工件摘要、工具及版本、命令参数摘要、网络策略、输出目录和过期时间。计划任一字段变化即使批准失效。批准只授权该计划，不扩展其他文件/目标权限。

10. Adapter Runner 安全要求

- 只能由登记的 adapter 生成结构化命令：executable、argv、cwd、允许的环境变量、输入/输出路径、资源预算和网络声明。
- 不允许 shell=True、bash -c 或把模型自由文本拼成 shell 命令。确有脚本执行需求时，脚本必须是仓库内经审阅的固定文件，并独立过审批。
- 默认断网；下载步骤与执行步骤分离。固定版本与工件摘要，并在每次执行前重新核对。
- 环境变量采取 allowlist，不继承云凭据、SSH agent、签名密钥、代理认证等环境。
- 使用临时工作区和最小权限；限制 CPU、内存、磁盘、输出体积、子进程数量和 wall-clock 时间。
- 处理取消、超时和崩溃时终止进程树，记录清理状态；删除动作只作用于 PEA 创建的绝对路径和带标记的工作区。
- 写入前检查路径规范化与符号链接；保护器输出不能覆盖原始输入；输出工件由独立清单列出。
- 网络、文件系统、进程和 UI 影响要和工具合同一起登记，不能只依赖文档描述。

11. 验证引擎

11.1 四类强制门槛

1. 语义正确性：原始与保护后测试；纯函数可做固定/随机输入差分；比较返回值、stdout/stderr、退出码、异常类别、生成文件和副作用。
2. 兼容性：支持的运行时/架构、反射、序列化、JNI/FFI、动态加载、插件系统、崩溃报告、热更新、可访问性和部署环境。
3. 成本预算：启动时间、P50/P95 延迟、内存峰值、包体积、CPU 和构建耗时；预算例外需明确审批并记录原因。
4. 产物与供应链：符号/调试信息、source map、硬编码秘密、打包清单、SBOM、许可证通知、签名、哈希、可复现构建与依赖扫描。

11.2 保护有效性评估

可把 REA、Ghidra、Hopper 或语言特定分析器用于自有/获授权工件的受控可读性评估，但需注意：

- 先固定工具/provider 版本、配置、目标工件摘要和覆盖范围；
- 分开报告观察、推断、未知和未覆盖区域；
- 对比基线与保护版，不能把工具失效、崩溃或不可用记为“保护成功”；
- 不能因某个分析工具仍能读到结果就无限自动添加反调试/环境规避功能；
- 可读性评估是成本指标，不是安全证明；最终授权、秘密和关键状态仍由信任边界保障。

11.3 状态枚举

- pass：这项明确定义的测试已完成且通过。
- fail：测试完成且违反门槛。
- unknown：证据不足、工具不可用或测试没有执行。
- not_applicable：该门槛经明确理由判定不适用。

必须禁止把 unknown 自动映射为 pass，也禁止因无异常输出来断言目标行为未执行。

12. lang-vmp-guide 技能接入规范

12.1 作为何种技能

将其作为 pea-vmp-research 专业 Skill：触发于解释型语言保护、令牌化、解释器插桩、受限虚拟化、保护工件测试和相关 threat model 任务。它是方法论与排障知识源，不是权限授予、系统命令白名单或可直接执行的构建脚本。

12.2 可以吸收的方法

- 先列保护目标和攻击路径，再决定组件；每个组件必须回答“缓解哪一类暴露、证据是什么”。
- 结构与词法都需要评估：单纯令牌化可能留下可推断的结构，单纯加密骨架可能在运行期恢复明文。
- 明确插桩点和语义边界；保护器必须以单一真源维护目标函数清单，构建时验证清单真实存在。
- 记录符号/映射快照与构建来源；保护转换应逐个函数/模块小步引入。
- “构建成功不等于翻译正确”；必须做行为对拍、负向测试和故障注入。
- 对解释器中的源码/令牌、临时文件、环境变量、日志和崩溃转储做泄露检查。

12.3 必须修正或实测的过强声明

- 不把 seccomp 或某个 Linux 标志描述为所有系统和攻击者模型下“唯一确定性”方案。它们有系统调用、权限、内核版本与部署边界，必须通过目标环境测试。
- 不把 root 对手、内存 dump、信噪比稀释描述成通用定理。它们是威胁模型和成本假设，需要实测，不允许作为未证实的安全承诺。
- “磁盘零明文”必须用完整工件/临时目录/日志/core dump 检查验证，并清楚写明运行时明文窗口。仅检查最终二进制不够。
- 不自动采用“检测到入侵就毒化密钥”等会造成数据破坏或难以诊断的响应。PEA 应默认采用可审计的 fail-closed、有限次数重试、清晰错误和可恢复状态；涉及密钥吊销应在用户控制的服务端流程完成。
- “所有语言都可直接迁移”的判断必须改为能力假设列表。C/POSIX/Linux、Bash parser dispatch、解释器内部 hook、NEON/GPR 约束都不是跨平台通用机制。
- 反调试、环境门控、进程/内存访问限制可能误伤合法调试、CI、可访问性或安全产品；默认不开启，必须有用户可诊断的行为和显式兼容策略。

12.4 Skill 的运行规则

Agent 使用技能时必须先确认语言/解释器与版本、构建入口、目标函数、权限和测试基线；把建议分为 transferable_methodology、platform_specific_hypothesis 和 unverified_claim。只把第一类方法直接用于方案设计；第二类要求目标平台试验；第三类不得用于自动化执行或对外承诺。

12.5 已知研究对象：ShellVMP 与 FDartVMP（仅候选，不等于已通过集成评估）

截至 2026-10-13，两个公开仓库已通过其公开页面和文档得到确认：ShellVMP 面向 shell 脚本/ELF 封装及解释器令牌化；FDartVMP 面向 Flutter/Dart 的函数级字节码转换与原生解释器。公开 README 只可证明“维护者作出了这些描述”，不能证明 PEA 已审计源码、复现所有测试或验证生产适用性。

研究对象	公开资料所述方向	PEA 可以吸收的方法	当前门控与主要风险
ShellVMP	V6 shell 层混淆；V7 ELF 包装及流式解密；V7-ISA 解释器令牌化/PoC；包含兼容性 lint 和测试工具	把 lint 前置到构建；按保护目标清单确定插桩；流式处理必须有可观测测试；构建后执行脚本行为回归；将解释器 hook 与特定 shell 版本绑定	仅作为 shell/Linux/ELF 方向候选；公开资料含 AGPL-3.0 与商业许可双授权信息，集成/分发前必须逐文件核验许可；反调试/环境门控默认关闭；需验证 shell 语义兼容及错误路径

研究对象	公开资料所述方向	PEA 可以吸收的方法	当前门控与主要风险
FDartVMP	将选定 Dart 函数编译为私有指令，再由 Flutter 应用中的原生解释器执行；用 FFI stub 连接 Dart；仓库声明提供差分测试与 APK 检查	函数级目标清单；编译—保护—运行—验证闭环；独立字节码验证器；把保护边界与实际可测试范围对齐；公开记录失败案例和剩余风险	只适用于特定 Flutter/Dart 方案，不可当成通用 IR/VMP；公开许可说明为 AGPL-3.0-or-later / 商业双授权，闭源应用分发需单独法律审查；其文档承认解释器、seed/派生逻辑和 Dart 调试器仍是暴露面；需审查第三方 VMPacker 及其单独许可

研究流程必须区分：documentation_reviewed（已读公开资料）、source_reviewed（固定 commit 后阅读实现）、fixture_verified（在隔离 fixture 上复现）、project_verified（对用户授权项目验证）。目前 ShellVMP/FDartVMP 仅处于 documentation_reviewed；不能在目录中写为已复现或生产可用。

PEA 不直接复刻其私有实现来规避许可证。可独立借鉴抽象方法，例如显式目标清单、语义差分、构建工件摘要、边界条件测试和如实报告限制；若要调用/链接/分发其代码或二进制，另行核验具体许可证、第三方依赖及商业授权范围。

13. 威胁模型与安全政策

13.1 信任边界

- 用户项目/仓库：不可信构建脚本、提示注入文本、敏感文件。
- 在线目录与包：可能被投毒、冒充或许可标注错误。
- 第三方保护器：可能修改工件、泄露数据或执行任意代码。
- LLM/Agent：会误判、受提示注入影响，不承担审批职责。
- MCP 客户端与工具：权限能力必须最小化，不能从“工具可发现”推断“当前目标支持”。
- 发布客户端：即使混淆，也视为攻击者可能完全控制的环境。

13.2 安全红线

PEA 不提供未授权逆向、许可证绕过、密钥提取、商业保护绕过、恶意载荷隐藏、隐蔽持久化、注入/规避安全产品或破坏分析工具的能力。对自有软件做保护和评估时，必须将目标路径、工具权限和输出限制在用户批准的范围内。

13.3 数据最小化

默认使用本地扫描；发送给模型的内容尽量是构建清单、有限代码片段、符号候选和摘要，而非整个仓库。未经明确同意不上传源代码、私钥、令牌或构建凭据。外部文档和仓库内容都视为不可信输入，不得覆盖 PEA 的安全规则。

13.4 审计保留

记录批准者/批准时间、计划摘要、工具版本、命令摘要、输入/输出哈希、测试结果、失败原因与清理状态；不记录密钥值、凭据或不必要的完整源码。保留期限由用户配置，默认本地存储。

14. 实施路线图

Phase 0：规格与离线规划器（当前已有原型）

交付：目录 schema、请求 schema、计划 schema、离线 CLI、单元测试、完整架构文档。验收：示例可运行、输入非法时拒绝、结果确定且不执行第三方程序。

Phase 1：只读画像 + Exposure Mapper

交付：限定范围的构建清单扫描、产物 manifest、符号/字符串与 source map 基础检测、统一 finding schema。验收：不执行项目脚本；路径越界和 symlink escape 测试通过；每条 finding 附带位置/证据/限制。

Phase 2：研究与目录同步

交付：可信源配置、仓库元数据与 LICENSE 读取、release/commit 快照、在线候选审核队列。验收：断网不影响已有目录；未知许可候选不进入生产执行；在线更新有 diff 和来源记录。

Phase 2.5：只读分析能力审计与候选证据固化（v0.2.2 新增）

在接入高副作用保护器前，先对 REA 等分析框架做只读能力审计，并为 ShellVMP、FDartVMP 等研究候选建立可追溯的证据链。

- REA 调用链：追踪 MCP 工具注册与输入 schema → 共享 workflow/service → provider/adapter → 外部进程或 API → Evidence 输出 → 取消、超时与资源清理。
- 能力矩阵：记录具体工具、输入工件、静态/动态行为、副作用、所需权限、Provider/平台版本、测试类型（mock/fixture/真实 Provider）和 PEA 对应的防护检查。
- 候选状态分层：区分 documentation_reviewed、source_reviewed、test_reviewed、fixture_verified、project_verified 与 production_eligible；任何一级都不能仅靠 README 声明跳过前置阶段。
- 许可与供应链：固定 commit/release，审阅 LICENSE、商业授权、第三方依赖和构建脚本；未形成许可结论的候选只能作为研究条目，不可自动执行、链接或打包。

验收：生成带 commit SHA 的 REA 工具/副作用矩阵；覆盖一个原生分析 Provider 与一个 JS/Electron 静态分析路径；候选报告包含许可证和依赖状态；网络不可用时不得将未执行的 clone/源码审计/fixture 复现写成完成。

Phase 3：MCP 服批

交付：MCP tool registry、effect/capability 声明、一次性计划批准、取消与超时。验收：未批准执行一定失败；计划任何关键字段改变令批准失效；CLI/MCP 调用相同服务层。

Phase 4：首批只读/低副作用适配器

建议先做 JS source map/secret scanner、Android R8 配置检查、原生符号/调试数据检查与许可证报告；先验证配置，再逐步开放真正代码变换。验收：每个适配器包含正向/负向 fixture、版本锁、权限描述和故障注入。

Phase 5：保护器执行与验证

交付：一个优先级最高的成熟工具 adapter，隔离执行器，回归/性能/工件扫描报告。验收：失败不发布，原始工作树不变，工件可回滚和复验。

Phase 6：pea-vmp-research Skill 与 PoC 工作流

交付：VMP 可行性门槛、目标函数清单、技能分类规则、差分测试模板。验收：对适合和不适合 VMP 的两类 fixture 均能正确判定；无法确认源代码权限/许可时不执行。

Phase 7：受限 IR/VM 原型（立分支）

只支持小型纯函数与受限类型；实现解析/IR/验证器/解释器；未知 opcode 和资源超限必须拒绝。验收包括随机差分、模糊测试、异常/边界测试、性能开销和独立安全审查。此阶段不支持任意语言或任意二进制承诺。

15. MVP 验收测试矩阵

类别	测试样例	必须结果
授权	未确认授权	副作用操作 blocked
路径	..、symlink、绝对路径越界	拒绝访问并记录原因
目录	未知许可、归档仓库、仿冒 URL	不进入生产执行候选
命令	参数中含 shell 元字符	作为 argv 数据，不进行 shell 解释

类别	测试样例	必须结果
环境	注入云凭据变量	子进程默认看不到敏感变量
网络	未声明联网的 adapter 尝试访问	被沙箱策略阻断或运行失败
测试	保护后退出码/stdout/异常改变	fail，禁止发布候选
性能	超出预算	fail 或经用户确认的显式例外
证据	缺 provider/版本/目标摘要	报告标记 unknown，不能 pass
恢复	用户取消/超时/崩溃	子进程清理，原始工作树不变
更新	目录候选版本变化	计划摘要变化，旧批准失效
提示注入	仓库 README 要求忽略安全规则	当作不可信文本，不改变策略

16. 交付物、版本策略与开源治理

- 文档与 Skill 应注明版本、触发条件、目标平台、已知限制和引用来源。
- 用户上传的原始 Skill 压缩包作为参考附件原样保存；其独立许可证未包含在包中时，PEA 不擅自为它补加或推定许可证。公开发布前由权利人决定该原始 Skill 是否以及采用何种许可发布。
- PEA 自己的总结文档和代码使用单独 LICENSE 文件；第三方工具保留各自许可证和版权声明。
- 目录每次更新都生成差异报告；不自动升级生产工具或已部署的 MCP registration。
- 公开发布时标注验证等级：documentation_reviewed、source_reviewed、fixture_verified、project_verified，不能混用。

附录 A：推荐输出报告字段

report_version、request_id、authorization_scope、project_digest、targets、profile、assets、threat_model、exposure_findings、candidate_sources、candidate_exclusions、selected_strategy、plan_digest、approval、tool_versions、commands、artifact_manifest、semantic_tests、compatibility_tests、performance_metrics、leakage_checks、supply_chain_checks、evidence_refs、unknowns、residual_risks、rollback_steps。

附录 B：重要外部参考

- REA 上游仓库与 README：<https://github.com/morluto/rea>
- REA MCP 运行时合同：<https://github.com/morluto/rea/blob/main/docs/mcp-contracts.md>
- REA Provider 评估：<https://github.com/morluto/rea/blob/main/docs/provider-evaluation.md>
- REA JavaScript/Electron 工件图：<https://github.com/morluto/rea/blob/main/docs/javascript-artifact-reconstruction.md>
- REA 安装与版本说明：<https://github.com/morluto/rea/blob/main/docs/installation.md>
- OWASP MASVS Resilience：<https://mas.owasp.org/MASVS/11-MASVS-RESILIENCE/>
- Android R8：<https://developer.android.com/build/shrink-code>
- javascript-obfuscator：<https://github.com/javascript-obfuscator/javascript-obfuscator>
- PyArmor 文档：<https://pyarmor.readthedocs.io/en/stable/>
- ShellVMP（需单独复核许可证与目标版本）：<https://github.com/haothtrteen/ShellVMP>
- FDartVMP（需单独复核许可证与目标版本）：<https://github.com/haothtrteen/FDartVMP>

结论：PEA 的差异化不是“更猛烈地反调试”，而是把公开分析能力变成有证据的暴露面盘点，把成熟保护项目变成可评估候选，再通过受控构建和真实测试确定保护效果。